

Five Minutes In The Bathroom
(ห้านาทีที่ห้องน้ำ)

จัดทำโดย

นายพิพัฒน์ โชคโคกกรวด รหัสนักศึกษา 6804062612277

เสนอ

ผู้ช่วยศาสตราจารย์ สถิตย์ ประสมพันธ์

วิชา 040613204 OBJECT-ORIENTED PROGRAMMING Section 2

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาคเรียนที่ 1 ปีการศึกษา 2569

บทที่ 1 บทนำ

ที่มาและความสำคัญของโปรเจ็ค

ที่มาเกิดจากที่ผมส่วนตัวเป็นคนที่ชอบเล่นเกมชอบดูพวกสื่อ Horror อยู่แล้ว และแรงบันดาลใจของโปรเจ็คนี้ส่วนใหญ่ได้มาจาก Five Nights at Freddy's เกมแนว Survival Horror ในรูปแบบ Point-and-Click ที่ผมชอบดูในตอนเด็กๆ แล้วนำมาผสมผสานกับการณ์ใกล้ตัวในชีวิตประจำวันของในหอพัก ซึ่งเป็นผลที่ผมจะได้ลองทำเกมที่ได้ประยุกต์จากสิ่งที่เรียน

ประเภทของโครงการ

เกมคอมพิวเตอร์สำหรับระบบปฏิบัติการเดสก์ท็อป ประเภท Survival Horror / Point-and-Click Strategy พัฒนาด้วยภาษา Java และเทคโนโลยี Java Swing GUI

ประโยชน์

- ช่วยฝึกไหวพริบในการเอาตัวรอด
- ช่วยฝึกในการตัดสินใจสิ่งต่างๆ
- ช่วยฝึกให้มีสมาธิในการจดจอ
- ช่วยการจัดลำดับความสำคัญ
- เพื่อความบันเทิง

ขอบเขตของโครงการ

Five Minutes In The Bathroom

(ห้านาทีที่ห้องน้ำ)

เนื้อหาเกี่ยวกับเกมส์

·รายละเอียดเกม

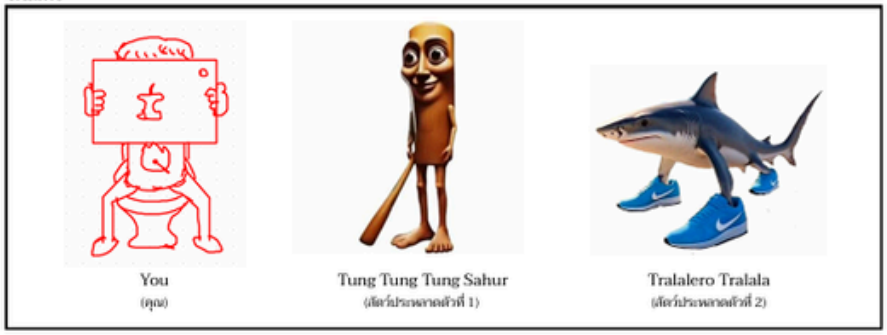
ในช่วงดึกคืนหนึ่ง ณ หอพักชาย คุณตื่นขึ้นมากลางดึกด้วยความรู้สึกปวดท้องอย่างรุนแรงจึงรีบวิ่งไปเข้าห้องน้ำ แต่ในระหว่างที่กำลังทำธุระส่วนตัวอยู่นั้น คุณก็ได้ยินเสียงแปลกๆที่ดังก้องมาจากชั้น 1 คุณหยิบแท็บเล็ตติดมือมาด้วย และเข้าแอปพลิเคชันในเครื่องสามารถเชื่อมต่อกับระบบกล้องวงจรปิด (CCTV) ของหอพักได้ คุณพบว่ามีสิ่งมีชีวิตประหลาด 2 ตัวกำลังเดินวนเวียนจากชั้น 1 ขึ้นมาชั้น 2 และเป้าหมายของพวกมันคือห้องน้ำที่คุณกำลังนั่งอยู่ ทางรอดเดียวจากที่คุณได้ยินมาคือ ใน 5 นาทีนั้นคุณต้องอย่าให้มันเข้ามาได้เป็นอันขาด! โดยใช้แท็บเล็ตสอดส่องพวกมันผ่านกล้อง และคอยหันไปล็อกประตูห้องน้ำหรือปิดฝาช่องระบายอากาศให้ทันก่อนที่พวกมันจะทะลวงเข้ามาถึงตัว!

·วิธีการเล่น

- เอาชีวิตรอดในห้องน้ำให้ครบ 5 นาที โดยป้องกันไม่ให้สัตว์ประหลาดทั้ง 2 ตัวบุกเข้ามาถึงตัวได้สำเร็จ โดยผู้เล่นจะต้องบริหารเวลาและสมาธิโดยการสลับหน้าจอไปมาระหว่าง 2 โหมดหลักคือ 1. มุมมองบุคคลที่หนึ่งขณะนั่งอยู่ในห้องน้ำ ผู้เล่นจะมองเห็นประตูห้องน้ำและ ช่องระบายอากาศ ที่ผู้เล่นเอาไว้ "ป้องกันตัว" 2. เมื่อกดปุ่มสลับโหมด ตัวละครจะกันลงมองแท็บเล็ตเพื่อเช็กกล้องวงจรปิดทั้ง 4 ตัว โดยสังเกตการณ์ว่าสัตว์ประหลาดกำลังเคลื่อนที่เข้ามาใกล้เราหรือยัง

·Storyboard

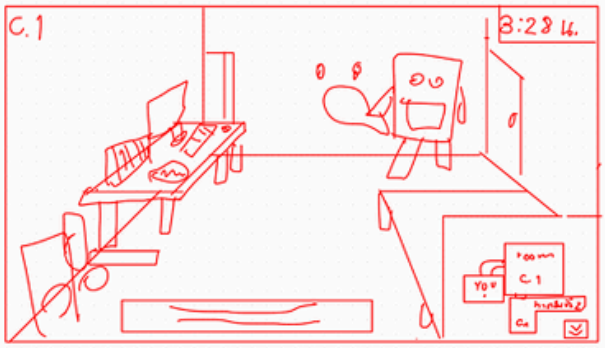
ตัวละคร



Main Menu

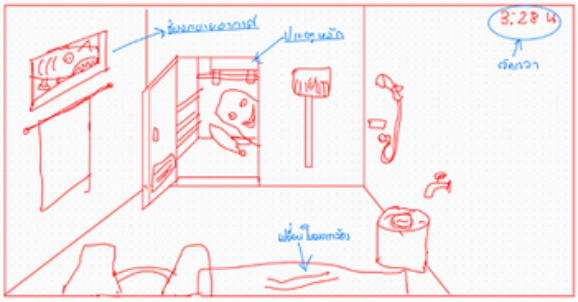


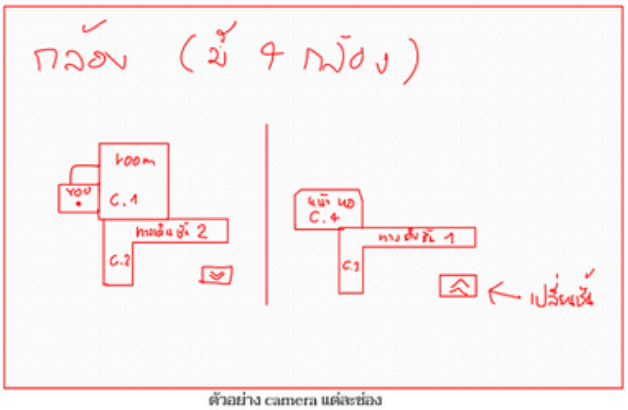
โหมดกล้อง



ตัวอย่าง camera 1

โหมดปกติ





ตัวอย่าง camera แต่ละช่อง

หน้า GameOver



หน้า GameWin



·ประโยชน์

1. ช่วยฝึกไหวพริบในการเอาตัวรอด
2. ช่วยฝึกในการตัดสินใจสิ่งต่างๆ
3. ช่วยฝึกให้มีสมาธิในการจดจอ
4. ช่วยการจัดลำดับความสำคัญ
5. เพื่อความบันเทิง

·ตารางแผนงานการทำงานว่าจะทำอะไรยังไงบ้างใน 2เดือนก่อนส่ง

ลำดับ	รายการ	สัปดาห์ที่ 1-2	สัปดาห์ที่ 3-4	สัปดาห์ที่ 5-6	สัปดาห์ที่ 7-8
1	ออกแบบโครงสร้างโค้ด (OOP), วางระบบ Game Loop, และสร้างหน้าต่าง UI โหมดปกติ/โหมดกล้อง ด้วย Java Swing				
2	พัฒนาระบบตัวนับเวลา 5 นาที, ระบบสลับหน้าจอกล้อง, และผสานภาพกราฟิก (รูปวาด) เข้าไปใน Frame				
3	เขียน Monster AI Logic ให้ศัตรูสุ่มเดินตาม State และเขียนระบบตรวจจับการชน (ปิดประตูกันทัน/ไม่ทัน)				
4	ใส่ระบบเสียง (Jumpscare, เสียงแบคกราวด์), ปรับสมดุลเกม, ทดสอบแก้ Bug, และ Build เป็นไฟล์ .jar เพื่อนำเสนอ				

บทที่ 2 ส่วนการพัฒนา

เนื้อเรื่องย่อหรือวิธีการเล่น

เนื้อเรื่อง: ในช่วงดึกคืนหนึ่ง ณ หอพักชาย คุณตื่นขึ้นมากลางดึกด้วยความรู้สึกปวดท้องอย่างรุนแรงจึงรีบวิ่งไปเข้าห้องน้ำ แต่ในระหว่างที่กำลังทำธุระส่วนตัวอยู่นั้น คุณก็ได้ยินเสียงแปลกๆที่ดังก้องมาจากชั้น 1 คุณหยิบแท็บเล็ตติดมือมาด้วย และเข้าแอปพลิเคชันในเครื่องสามารถเชื่อมต่อกับระบบกล้องวงจรปิด (CCTV) ของหอพักได้ คุณพบว่ามีสิ่งมีชีวิตประหลาด 2 ตัวกำลังเดินวนเวียนจากชั้น 1 ขึ้นมาชั้น 2 และเป้าหมายของพวกมันคือห้องน้ำที่คุณกำลังนั่งอยู่ ทางรอดเดียวจากที่คุณได้ยินมาคือ ใน 5 นาทีนั้นคุณต้องอย่าให้มันเข้ามาได้เป็นอันขาด! โดยใช้แท็บเล็ตสอดส่องพวกมันผ่านกล้อง และคอยหันไปล็อกประตูห้องน้ำหรือปิดฝาช่องระบายอากาศให้ทันก่อนที่พวกมันจะทะลวงเข้ามาถึงตัว!

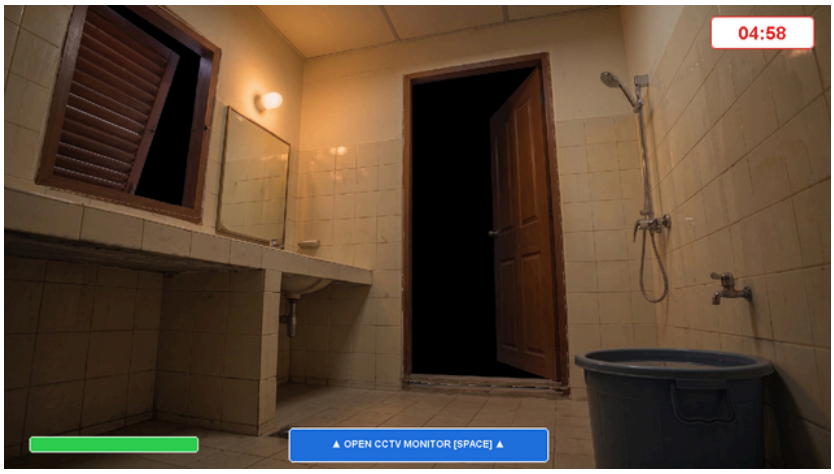
วิธีการเล่น: เอาชีวิตรอดในห้องนำให้ครบ 5 นาที โดยป้องกันไม่ให้สัตว์ประหลาดทั้ง 2 ตัวบุกเข้ามาถึงตัวได้สำเร็จ โดยวิธีปิดคือ

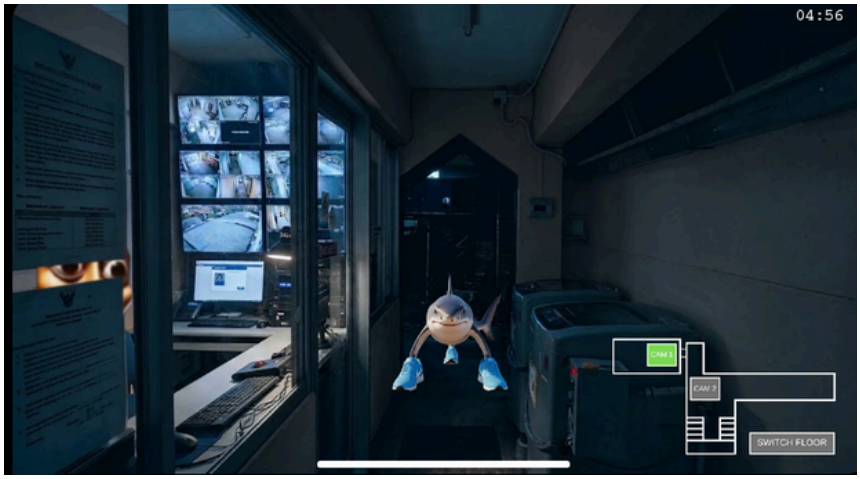
- ปิดประตูกั้น กดปุ่ม [E] หรือเมาส์คลิกที่ประตูก็ได้ เมื่อ Sahur อยู่หน้าประตู
- ปิดช่องลมกั้น กดปุ่ม [Q] หรือเมาส์คลิกที่ช่องลมก็ได้ เมื่อ Tralala อยู่หน้าช่องระบายอากาศ

ปิดค้างไว้จนกว่าพวกมันจะยอมถอยกลับไปและอย่าปิดนานเกินไป ถ้า STAMINA หมดจะปิดไม่ได้

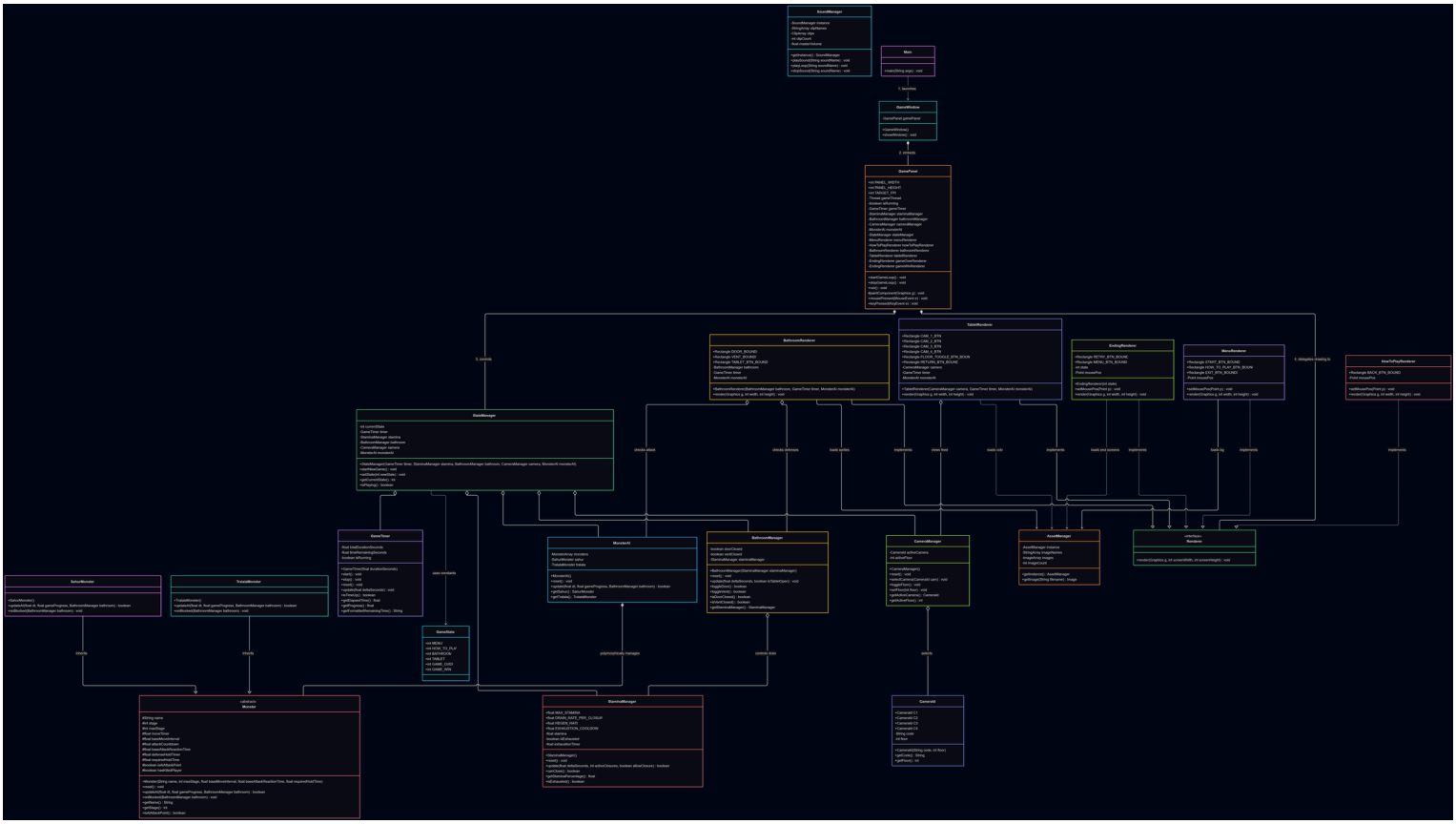
แล้วคุณสามารถเป็น กล้อง CCTV ได้ผ่านการกด [SPACE] หรือเมาส์คลิกที่ปุ่มสีฟ้าข้างล่างสุด

โดยโหมด CCTV สามารถติดตามการเดินของสัตว์ประหลาดได้ผ่านกล้องตัวต่างๆ โดยการคลิกเมาส์ที่ช่องนั้น และการฟังเสียงเป็นอีกอย่างหนึ่งที่สำคัญเหมือนกัน





คลาสไดอะแกรม พร้อมคำอธิบาย



คำอธิบาย

1. **Main:** คลาสเริ่มต้นของโปรแกรม ทำหน้าที่รันคำสั่งบน Event Dispatch Thread (EDT) ของ Java Swing ผ่านคลาสไม่ระบุชื่อ Runnable เพื่อสร้างอ็อบเจกต์หน้าต่าง GameWindow
2. **GameWindow:** คลาสหน้าต่างหลักของเกมที่สืบทอดมาจาก JFrame กำหนดขนาดหน้าจอคงที่ 1280x720 พิกเซล, ปิดการปรับขนาดหน้าจอ, จัดกึ่งกลางหน้าจออัตโนมัติ และตรวจจับการปิดหน้าต่างเพื่อสั่งหยุด Game Loop
3. **GamePanel:** คลาสหลักในการรันเกมที่สืบทอดมาจาก JPanel และอิมพลีเมนต์ Runnable, MouseListener, MouseMotionListener, KeyListener ทำหน้าที่:
   - ควบคุม Game Loop ที่ความถี่ 60 FPS คงที่ (Fixed Timestep) ผ่าน Thread และคำสั่ง Thread.sleep()
   - รวมรวมอ็อบเจกต์ระบบย่อยทั้งหมดผ่านความสัมพันธ์แบบ Composition
   - รับและกระจายสัญญาณอินพุตจากเมาส์และแป้นพิมพ์ ([E], [Q], [Space], [ESC])
   - วาดผลหน้าจอผ่านการ Override เมธอด paintComponent(Graphics g)
4. **GameState:** คลาสเก็บค่าคงที่สถานะของหน้าจอเกมในรูปแบบ public static final int
5. **StateManager:** คลาสจัดการสถานะและการเปลี่ยนฉากของเกม ควบคุมการเริ่มเกมใหม่ การรีเซ็ตค่าตัวแปรของระบบย่อยทั้งหมด และการสั่งตัดเสียงบรรยากาศเมื่อเข้าสู่ฉากจบเกม
6. **Monster:** คลาสแม่ Abstract Class ของมอนสเตอร์ทั้งหมด มีแอตทริบิวต์ protected สำหรับเก็บค่าสถานะ เช่น ระดับการเดิน (stage), เวลานับถอยหลังในการเดิน (moveTimer), เวลาสำหรับให้ผู้เล่นตั้งตัวป้องกัน (attackCountdown) และเวลาที่ผู้เล่นต้องปิดค้างเพื่อไล่มอนสเตอร์ (requiredHoldTime) พร้อมกำหนด Abstract Method สองตัวคือ updateAI() และ onBlocked()

7. **SahurMonster:** คลาสลูกที่สืบทอดมาจาก Monster รับผิดชอบพฤติกรรมของมอนสเตอร์ตัวที่ 1 ซึ่งจะสุ่มเดินผ่านกล้อง 4 ตัว (CAM 1 → 2 → 3 → 4) จนมาถึงหน้าประตูห้องน้ำ
8. **TralalaMonster:** คลาสลูกที่สืบทอดมาจาก Monster รับผิดชอบพฤติกรรมของมอนสเตอร์ตัวที่ 2 ซึ่งจะสุ่มมุดท่อระบายอากาศจากบริเวณกล้อง CAM 1 มายังช่องระบายอากาศของห้องน้ำโดยตรง
9. **GameTimer:** จัดการเวลานับถอยหลัง 5 นาที (300 วินาที) ของเกม มีเมธอดแปลงเวลาที่เหลือให้อยู่ในรูปข้อความดิจิทัล "MM:SS" เพื่อนำไปแสดงผลบนนาฬิกาในห้องน้ำและแท็บเล็ต
10. **StaminaManager:** จัดการระบบพลังกายของผู้เล่น (0–100 หน่วย) คำนวณอัตราการลดลงเมื่อปิดประตูหรือหน้าต่าง (10 หน่วย/วินาที/บาน) การฟื้นฟูพลังเมื่อปล่อยมือ (15 หน่วย/วินาที) และควบคุมระยะเวลาหมดแรง (Exhaustion Cooldown: 3.5 วินาที)
11. **BathroomManager:** ควบคุมสถานะทางกายภาพของห้องน้ำ (ประตูเปิด/ปิด, ช่องลมเปิด/ปิด) โดยทำงานร่วมกับ StaminaManager เพื่อปฏิเสธการปิดป้องกันเมื่อผู้เล่นหมดแรง และบังคับเปิดประตูกะทันหันหากแรงหมดระหว่างที่ปิดค้างอยู่
12. **CameraId:** คลาสแทนอัตลักษณ์ของกล้องวงจรปิดแต่ละตัว กำหนดรหัสกล้อง (code) และชั้นที่กล้องติดตั้งอยู่ (floor) พร้อมประกาศตัวแปรคงที่สำหรับกล้องทั้ง 4 ตัว (C1, C2, C3, C4)
13. **CameraManager:** ควบคุมการทำงานของระบบกล้องวงจรปิด บันทึกตำแหน่งกล้องที่กำลังเปิดดูอยู่ (activeCamera), การสลับชั้นแสดงผลระหว่างชั้น 1 และชั้น 2 (toggleFloor)
14. **MonsterAI:** ประสานงานและขับเคลื่อนมอนสเตอร์ทั้งหมด จัดเก็บมอนสเตอร์ในรูปอาเรย์ Monster[] monsters เพื่อ loop อัปเดตตรรกะของมอนสเตอร์ทุกตัวพร้อมกันในแต่ละเฟรม
15. **AssetManager:** ออกแบบตามรูปแบบ Singleton Pattern ทำหน้าที่โหลดและแคชไฟล์รูปภาพ (.png, .jpg) เข้าสู่หน่วยความจำด้วยคลาส ImageIcon และ java.awt.Image โดยใช้อาเรย์ 1 มิติร่วมกับอัลกอริทึม Linear Search
16. **SoundManager:** ออกแบบตามรูปแบบ Singleton Pattern ทำหน้าที่โหลด จัดเก็บ และควบคุมการเล่นไฟล์เสียง .wav ด้วยระบบมัลติเธรด (Thread & Runnable) และอ่านค่าระดับเสียงเดซิเบลจากไฟล์ volume_config.txt ด้วยคลาส Scanner
17. **Renderer:** ส่วนต่อประสานชนิด Interface กำหนดแม่แบบเมธอด public abstract void render(Graphics g, int screenWidth, int screenHeight) เพื่อบังคับให้ทุกหน้าจอแสดงผลมีมาตรฐานการวาดภาพเดียวกัน
18. **MenuRenderer:** อิมพลีเมนต์ Renderer เพื่อวาดหน้าจอหลัก (Main Menu): วาดภาพพื้นหลัง, ข้อความชื่อเกมแบบเว้นบรรทัดสไตล์เกมสยองขวัญ, เมนูตัวอักษร (Start, How To Play, Exit) และลูกศร >> เมื่อนำเมาส์ไปชี้
19. **HowToPlayRenderer:** อิมพลีเมนต์ Renderer เพื่อวาดหน้าคู่มือการเล่น: จัดวางหัวข้อเป้าหมาย, วิธีเอาชีวิตรอด, แถบเตือนพลังกาย, ตารางปุ่มควบคุม และปุ่มย้อนกลับสู่เมนูหลัก
20. **BathroomRenderer:** อิมพลีเมนต์ Renderer เพื่อวาดมุมมองบุคคลที่หนึ่งภายในห้องน้ำ: ภาพพื้นหลังห้องน้ำ, สไปรต์ประตู (เปิด/ปิด/มอนสเตอร์โผล่), สไปรต์ช่องระบายอากาศ, กล่องนาฬิกานับถอยหลัง, หลอดพลังกายสีเขียว (หรือสีแดงกะพริบเมื่อหมดแรง) และปุ่มสลับแท็บเล็ต
21. **TabletRenderer:** อิมพลีเมนต์ Renderer เพื่อวาดหน้าจอมอนิเตอร์กล้อง CCTV: เลือกภาพกล้อง 1 ใน 10 รูปมาแสดงผลตามตำแหน่งจริงของมอนสเตอร์, วาดจุดไฟสีแดง REC กะพริบ และเวลานับถอยหลัง
22. **EndingRenderer:** อิมพลีเมนต์ Renderer เพื่อวาดหน้าจอจบเกม: ทั้งหน้าพ่ายแพ้ (GAME_OVER) และหน้าชนะเมื่อรอดครบ 5 นาที (GAME_WIN) พร้อมไฮไลต์ปุ่ม Retry และ Main Menu เมื่อผู้เล่นเลื่อนเมาส์ไปวางทับ

รูปแบบการพัฒนา Application

- ภาษาที่ใช้: Java (JDK 17+)
- ส่วนติดต่อผู้ใช้ (GUI): javax.swing (JFrame, JPanel)
- ระบบกราฟิก: java.awt (Graphics, Image, Color, Font)
- ระบบการทำงานของเกม: java.lang.Thread และ Runnable (Game Loop 60 FPS)
- ระบบเสียง: javax.sound.sampled (เล่นไฟล์เสียง .wav)
- โปรแกรมเตรียมภาพ: Canva,Gemini
- รูปแบบผลงาน: Executable JAR File (.jar) และ Source Code

อธิบายส่วนของโปรแกรมที่มี

Constructor

```java
// ไฟล์: src/entity/Monster.java
public Monster(String name, int maxStage, float baseMoveInterval,
               float baseAttackReactionTime, float requiredHoldTime) {
    this.name = name;
    this.maxStage = maxStage;
    this.baseMoveInterval = baseMoveInterval;
    this.baseAttackReactionTime = baseAttackReactionTime;
    this.requiredHoldTime = requiredHoldTime;
    reset(); // กำหนดค่าเริ่มต้นให้กับตัวแปรสถานะ
}
```

```java
// ไฟล์: src/core/GameWindow.java
public GameWindow() {
    super("Five Minutes In The Bathroom"); // เรียก Constructor ของ JFrame
    this.gamePanel = new GamePanel();
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setResizable(false);
    add(gamePanel);
    pack(); // ปรับขนาดหน้าต่างให้พอดีกับ GamePanel (1280x720)
    setLocationRelativeTo(null); // จัดกึ่งกลางหน้าจอ
}
```

- Constructor ทำหน้าที่กำหนดค่าเริ่มต้นให้แก่อ็อบเจกต์ทันทีที่ถูกสร้างขึ้น เช่น ในคลาส Monster จะรับพารามิเตอร์เพื่อกำหนดค่าความยากพื้นฐานและเรียกเมธอด reset() และในคลาส GameWindow จะเรียก super(...) ของคลาสแม่ JFrame พร้อมกำหนดขนาดและจัดหน้าต่างให้อยู่กึ่งกลางหน้าจอโดยอัตโนมัติ

Encapsulation

```java
// ไฟล์: src/system/StaminaManager.java
public class StaminaManager {
    private float stamina;          // ป้องกันการเข้าถึงตัวแปรโดยตรงจากภายนอก
    private boolean isExhausted;

    // ตรวจสอบความถูกต้องของเงื่อนไขก่อนอนุญาตให้ปิดประตู
    public boolean canClose() {
        return !isExhausted && stamina > 0.0f;
    }

    public float getStaminaPercentage() {
        return stamina / MAX_STAMINA;
    }
}
```

- ใช้หลักการซ่อนข้อมูลโดยกำหนดแอตทริบิวต์สำคัญ เช่น ค่าพลังกาย (stamina) และสถานะหมดแรง (isExhausted) ให้เป็น private แล้วเปิดให้คลาสภายนอกเข้าถึงหรือตรวจสอบผ่าน Getter Method (เช่น canClose(), getStaminaPercentage()) เพื่อป้องกันไม่ให้คลาสอื่นมาแก้ไขค่าพลังกายโดยตรงโดยไม่ผ่านเงื่อนไขความถูกต้อง

Composition

```java
// ไฟล์: src/core/GamePanel.java
public class GamePanel extends JPanel {
    // GamePanel ประกอบขึ้นจากอ็อบเจกต์ของระบบย่อยต่างๆ (Has-a relationship)
    private final GameTimer gameTimer;
    private final StaminaManager staminaManager;
    private final BathroomManager bathroomManager;
    private final CameraManager cameraManager;
    private final MonsterAI monsterAI;
    private final StateManager stateManager;

    public GamePanel() {
        this.gameTimer = new GameTimer(300.0f);
        this.staminaManager = new StaminaManager();
        this.bathroomManager = new BathroomManager(staminaManager);
        this.cameraManager = new CameraManager();
        this.monsterAI = new MonsterAI();
        this.stateManager = new StateManager(gameTimer, staminaManager, bathroomManager, cameraManager,
monsterAI);
    }
}
```

- คลาสหลัก GamePanel ถูกสร้างขึ้นโดยการรวบรวมอ็อบเจกต์ของระบบย่อยเข้ามาเป็นสมาชิกภายใน (ความสัมพันธ์แบบ "Has-a") ทำให้ GamePanel สามารถควบคุมวงจรชีวิตและสั่งการ GameTimer, StaminaManager, BathroomManager และ MonsterAI ให้ทำงานประสานกันได้อย่างเป็นระบบ

Polymorphism

```java
// ไฟล์: src/system/MonsterAI.java
public class MonsterAI {
    // จัดเก็บมอนสเตอร์ต่างชนิดกันไว้ในอาเรย์ของคลาสแม่ (Monster[])
    private final Monster[] monsters;

    public MonsterAI() {
        monsters = new Monster[2];
        monsters[0] = new SahurMonster();  // Upcasting
        monsters[1] = new TralalaMonster(); // Upcasting
    }

    public boolean update(float dt, float gameProgress, BathroomManager bathroom) {
        for (int i = 0; i < monsters.length; i++) {
            // เรียกเมธอด updateAI เดียวกัน แต่มอนสเตอร์แต่ละตัวจะทำงานตามพฤติกรรมเฉพาะของตัวเอง
            boolean killed = monsters[i].updateAI(dt, gameProgress, bathroom);
            if (killed) return true;
        }
        return false;
    }
}
```

- คลาส MonsterAI ใช้อาเรย์พ้องรูป Monster[] เก็บอ็อบเจกต์ของ SahurMonster และ TralalaMonster ผ่านการ Upcasting เมื่อวนลูปเรียกเมธอด updateAI() ระบบจะเลือกประมวลผลตรรกะการเดินเฉพาะตัวของมอนสเตอร์แต่ละตัวโดยอัตโนมัติ (Dynamic Binding) ทำให้สามารถเพิ่มมอนสเตอร์ตัวใหม่ได้ง่าย

Abstract Class

```java
// ไฟล์: src/entity/Monster.java
public abstract class Monster {
    protected final String name;
    protected int stage;
    protected boolean isAtAttackPoint;

    // Abstract Method บังคับให้คลาสลูกต้องนำไปเขียนการทำงานเอง
    public abstract boolean updateAI(float dt, float gameProgress, BathroomManager bathroom);
    public abstract void onBlocked(BathroomManager bathroom);
}
```

- กำหนดให้คลาส Monster เป็น abstract class ทำหน้าที่เป็นพิมพ์เขียวแม่แบบ ไม่สามารถสร้างเป็นอ็อบเจกต์ได้โดยตรง โดยประกาศ Abstract Method เช่น updateAI() และ onBlocked() เพื่อบังคับให้คลาสลูกที่จะสืบทอดไปต้องเขียนโค้ดรองรับพฤติกรรมการเคลื่อนที่และการถูกไล่ล่าเฉพาะตัวเสมอ

Inheritance

```java
// ไฟล์: src/entity/SahurMonster.java
public class SahurMonster extends Monster {

    public SahurMonster() {
        // สืบทอดคุณสมบัติและส่งค่าพื้นฐานไปยังคลาสแม่ผ่าน super(...)
        super("Tung Tung Tung Sahur", 4, 12.0f, 3.8f, 2.5f);
    }

    @Override
    public boolean updateAI(float dt, float gameProgress, BathroomManager bathroom) {
        // ใช้งานตัวแปร stage, moveTimer ที่สืบทอดมาจากคลาสแม่ Monster
        if (!isAtAttackPoint) {
            moveTimer -= dt;
            // ... ตรรกะการเดินเฉพาะตัวของ Sahur ...
        }
        return false;
    }
}
```

- คลาส SahurMonster และ TralalaMonster สืบทอดคุณสมบัติ (extends) มาจากคลาสแม่ Monster ทำให้สามารถนำตัวแปรสถานะพื้นฐาน (เช่น stage, moveTimer, attackCountdown) มาใช้งานได้ทันทีโดยไม่ต้องเขียนซ้ำ และทำการ @Override เมธอดเพื่อเพิ่มพฤติกรรมเฉพาะทาง

โครงสร้าง GUI และส่วนประกอบ Component

```java
// ไฟล์: src/core/GamePanel.java
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);

    int current = stateManager.getCurrentState();
    switch (current) {
        case GameState.MENU:
            menuRenderer.render(g, getWidth(), getHeight());
            break;
        case GameState.BATHROOM:
            bathroomRenderer.render(g, getWidth(), getHeight());
            break;
        case GameState.TABLET:
            tabletRenderer.render(g, getWidth(), getHeight());
            break;
        case GameState.GAME_OVER:
            gameOverRenderer.render(g, getWidth(), getHeight());
            break;
    }
}
```

- โครงสร้างส่วนติดต่อผู้ใช้ (GUI) ของเกมสร้างขึ้นจาก 2 คอมโพเนนต์หลัก:

- **JFrame (GameWindow):** หน้าต่างโปรแกรมหลักของระบบ ขนาด 1280x720 พิกเซล
- **JPanel (GamePanel):** ผืนผ้าใบแสดงผลที่เปิดใช้งาน Double Buffering เพื่อภาพที่ลื่นไหล โดย Override เมธอด paintComponent(Graphics g) เพื่อสลับการวาดฉากต่างๆ ตามสถานะของเกม (เมนู, ห้องน้ำ, แท็บเล็ต, ฉากจบ) โดยใช้คำสั่งวาดภาพ g.drawImage(...), กล่องสี่เหลี่ยม g.drawRoundRect(...) และข้อความ g.drawString(...)

Event Handling

```java
// ไฟล์: src/core/GamePanel.java
@Override
public void keyPressed(KeyEvent e) {
    int key = e.getKeyCode();
    int current = stateManager.getCurrentState();

    if (current == GameState.BATHROOM) {
        if (key == KeyEvent.VK_E) {
            boolean closed = bathroomManager.toggleDoor(); // กด [E] สลับเปิด/ปิดประตู
            SoundManager.getInstance().playSound(closed ? "door_close" : "door_open");
        } else if (key == KeyEvent.VK_Q) {
            boolean closed = bathroomManager.toggleVent(); // กด [Q] สลับเปิด/ปิดช่องลม
            SoundManager.getInstance().playSound(closed ? "vent_close" : "vent_open");
        } else if (key == KeyEvent.VK_SPACE) {
            stateManager.setState(GameState.TABLET);       // กด [Space] เปิดแท็บเล็ต
        }
    }
}

@Override
public void mousePressed(MouseEvent e) {
    Point p = e.getPoint();
    // ตรวจจับการคลิกเมาส์บนปุ่มกล้อง CCTV บนหน้าจอแท็บเล็ต
    if (TabletRenderer.CAM_1_BTN.contains(p)) {
        cameraManager.selectCamera(CameraId.C1);
    }
}
```

- ระบบรับสัญญาณอินพุตจากผู้เล่นผ่าน 2 Listener หลักของ Java AWT:

- **KeyListener:** ดักจับการกดแป้นพิมพ์ เช่น ปุ่ม [E] ปิดประตู, [Q] ปิดช่องลม, [Spacebar] สลับเปิดดูแท็บเล็ต และ [ESC] ย้อนกลับเมนู
- **MouseListener:** ดักจับการคลิกเมาส์ผ่านเมธอด mousePressed() โดยนำพิกัดจุดที่คลิก (Point) มาตรวจสอบการชนกับขอบเขตสี่เหลี่ยมของปุ่ม (Rectangle.contains(p)) เช่น ปุ่มเลือกกล้อง CAM 1–4 หรือปุ่ม Start ในหน้าเมนู

อัลกอริทึมที่สำคัญในโปรแกรม

```java
// ไฟล์: src/core/GamePanel.java (Fixed Timestep 60 FPS Game Loop)
@Override
public void run() {
    long lastLoopTime = System.nanoTime();
    while (isRunning) {
        long now = System.nanoTime();
        float deltaSeconds = (now - lastLoopTime) / 1000000000.0f;
        lastLoopTime = now;

        updateGame(deltaSeconds); // คำนวณตรรกะเกม
        repaint();                // วาดกราฟิกใหม่

        // หน่วงเวลาให้อัตราเฟรมอยู่ที่ 60 FPS คงที่
        long frameTime = System.nanoTime() - now;
        long sleepTime = (OPTIMAL_TIME - frameTime) / 1000000;
        if (sleepTime > 0) Thread.sleep(sleepTime);
    }
}
```

```java
// ไฟล์: src/entity/SahurMonster.java (Monster AI State & Probability)
moveTimer -= dt;
if (moveTimer <= 0.0f) {
    moveTimer = currentInterval * (0.8f + (float) Math.random() * 0.4f);
    double roll = Math.random();
    if (roll < 0.70) {
        stage++; // 70% โอกาสเดินหน้าเข้าใกล้ห้องน้ำ
        if (stage >= maxStage) isAtAttackPoint = true;
    } else if (roll > 0.90 && stage > 0) {
        stage--; // 10% โอกาสเดินถอยหลังกลับไป
    }
}
```

- โปรแกรมมี 2 อัลกอริทึมหลักที่ใช้:

1. อัลกอริทึม Game Loop (Fixed Timestep 60 FPS): ควบคุมการอัปเดตตรรกะและการวาดผลหน้าจอให้ทำงานคงที่ 60 ครั้งต่อวินาที โดยคำนวณระยะเวลาเฟรมจริง (deltaSeconds) ทำให้การเคลื่อนที่และนับเวลาในเกมมีความแม่นยำเท่ากันในทุกเครื่องคอมพิวเตอร์
2. อัลกอริทึมสุ่มการเดินของ AI (Probability & Difficulty Scaling): ควบคุมพฤติกรรมศัตรู โดยใช้เวลาถอยหลังร่วมกับการสุ่มความน่าจะเป็น (Math.random()) เพื่อตัดสินใจเดินหน้าหรือถอยหลัง พร้อมทั้งปรับลดระยะเวลาการเดินลงเรื่อยๆ ตามความคืบหน้าของคืน ทำให้เกมมีความตื่นเต้นและยากขึ้นตามลำดับเวลา

บทที่ 3 สรุป

ปัญหาที่พบระหว่างการพัฒนา

- ในบางระบบหรือฟังก์ชันที่จะทำให้เกมสมบูรณ์มากขึ้น ไม่ได้นำมาใส่เพราะติดปัญหาเรื่องจัดการจัดสรรเวลา ทำให้เหลือเวลาพัฒนาเกมน้อยลง จึงต้องตัดบางระบบอย่างเช่น Jump Scare และระบบที่เพิ่มความท้าทายยิ่งคืนสูงขึ้นมอเตอร์ยิ่งเก่งขึ้น เป็นต้น
- ปัญหาเรื่องการ Balance Monster ว่าเราจะทำยังไงให้มีความท้าทายและสมเหตุสมผล
- ขีดจำกัดด้านความสามารถในการทำภาพหรือวาดภาพที่ไม่มาพอ จึงอาจจะต้องใช้ generative AI มาช่วยที่ซึ่งส่วนตัวผมชอบงานทำมือที่ทำด้วยฝีมือมนุษย์คนมากกว่า
- ขีดจำกัดด้านการเรียนรู้ที่อาจจะใช้เวลาเรียนรู้ที่นานเกินไป จึงต้องใช้ AI มาช่วย
- ปัญหาการด้านการวาดภาพให้ตรง
- ปัญหาการไฟล์ภาพและเสียงไม่ขึ้น

จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

- เกมแนว Survival Horror ที่ทำให้คุณ กลัว ตลก งง สงสัย ได้ในเวลาเดียวกัน
- การควบคุมแบบไร้ปุ่มสัมผัส
- ระบบการเล่น 2 มุมมอง

คำแนะนำสำหรับผู้สอนที่อยากให้อธิบาย หรือที่เรียนแล้วไม่เข้าใจ หรืออยากให้เพิ่มสำหรับน้อง ๆ รุ่นต่อไป

- ถ้าเป็นไปได้อยากให้มี Grader ที่โจทย์อาจจะง่ายหรือยากกว่า Quiz1 ไว้ให้ทำเป็นตัวทดลองหรือแนวทางในการสอบ Quiz1 สำหรับน้องๆ รุ่นต่อไป